

Day 2 – JavaScript & jQuery Training Documentation

JavaScript Introduction & Execution

What is JavaScript?

JavaScript is a **lightweight, interpreted programming language** used to create dynamic web applications.

Where JavaScript Runs

- Browser (Frontend)
- Server (Node.js)

How JS Executes

1. Code is parsed
2. Execution context is created
3. Code is executed line by line

```
<script>  
  console.log("Hello JavaScript");  
</script>
```

Variables, Data Types & Best Practices

Variables

```
var x = 10;      // avoid  
let y = 20;      // use  
const z = 30;    // preferred
```

Data Types

- Primitive: Number, String, Boolean, Undefined, Null
- Non- Primitive: Object, Array, Function

```
let userName = "Rahul";  
let marks = 85;  
let isActive = true;
```

Best Practices - Use const by default - Avoid global variables

Operators & Control Statements

Operators

- Arithmetic
- Comparison (== vs ===)
- Logical

```
if (10 === "10") {  
  console.log("Strict check");  
}
```

Conditions & Loops

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Functions (Deep & Optimized)

Normal Function

```
function add(a, b) {  
  return a + b;  
}
```

Arrow Function

```
const multiply = (a, b) => a * b;
```

Why Functions Matter

- Code reuse
- Testability
- Clean architecture

Arrays & Objects (Industry Usage)

Arrays

```
let numbers = [10, 20, 30];  
numbers.push(40);
```

Important Array Methods

```
let evenNumbers = numbers.filter(n => n % 2 === 0);  
let squared = numbers.map(n => n * n);
```

Objects

```
let employee = {  
  id: 1,  
  name: "Amit",  
  role: "Developer"  
};
```

DOM & Query Selectors (Very Important)

What is DOM?

DOM represents HTML as a tree of objects.

getElement vs querySelector

```
document.getElementById("title");  
document.querySelector("#title");  
document.querySelectorAll(".item");
```

DOM Manipulation

```
document.querySelector(".box").innerText = "Updated";
```

Events & Event Handling

```
document.querySelector("#btn").addEventListener("click", () => {  
  alert("Button clicked");  
});
```

Common Events

- click
 - change
 - keyup
 - submit
-

Scope, Hoisting & Closures

Scope Types

- Global
- Function
- Block

Hoisting

Variables are hoisted but not initialized.

```
console.log(a);  
var a = 10;
```

Closures

```
function counter() {  
  let count = 0;  
  return function () {  
    count++;  
    console.log(count);  
  }  
}
```

Asynchronous JavaScript (Must Know)

Callbacks

```
setTimeout(() => {  
  console.log("Callback executed");  
}, 1000);
```

Promises

```
let promise = new Promise((resolve, reject) => {  
  resolve("Success");  
});
```

```
promise.then(result => console.log(result));
```

Async & Await (Clean Async Code)

```
async function loadData() {  
  try {  
    let response = await fetch("/api/data");  
    let data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.log(error);  
  }  
}
```

Why async/await? - Readable code - Better error handling

jQuery –

Introduction to jQuery

What is jQuery?

jQuery is a **fast, lightweight JavaScript library** that simplifies:

- DOM manipulation
- Event handling
- AJAX calls
- Animations

Why jQuery Was Created

- Write less JavaScript code
- Handle cross-browser issues

When jQuery Is Used Today

- Legacy applications
- Existing enterprise projects
- Quick UI scripting

jQuery Syntax & Selectors

Basic Syntax

```
$(selector).action();
```

ID, Class & Element Selectors

```
$("#btn");  
$(".box");  
$("p");
```

Attribute & Multiple Selectors

```
$("input[type='text']");  
$("h1, h2, h3");
```

When to Use: Selecting DOM elements easily

Document Ready & Traversing

Ensures DOM is fully loaded.

```
$(document).ready(function () {  
    console.log("DOM Ready");  
});
```

DOM Traversing

```
$(".child").parent();  
$(".container").find("p");
```

Use Case: Navigating HTML structure

DOM Manipulation

Get / Set Content

```
$("#title").text();  
$("#title").text("New Title");
```

HTML & Value

```
$("#box").html("<b>Hello</b>");  
$("#input").val("Test");
```

CSS Manipulation & Classes

```
$(".box").css("color", "red");  
$(".box").addClass("active");  
$(".box").removeClass("active");
```

Best Practice: Prefer CSS classes over inline styles

Events & Event Handling

Basic Events

```
$("#btn").click(function () {  
    alert("Clicked");  
});
```

Common Events

- click
- change
- keyup

- submit

Event Object

```
$("#form").submit(function (e) {  
    e.preventDefault();  
});
```

Event Delegation

```
$(document).on("click", ".dynamic-btn", function () {  
    alert("Dynamic button clicked");  
});
```

When to Use:

- Dynamic elements
- Performance optimization

Effects & Animations

```
$(".box").hide();  
$(".box").fadeOut();  
$(".box").slideToggle();
```

Custom Animation

```
$(".box").animate({ width: "200px" });
```

jQuery AJAX (Very Important)

AJAX GET Request

```
$.ajax({  
    url: "/api/users",  
    method: "GET",  
    success: function (data) {  
        console.log(data);  
    },  
    error: function (err) {  
        console.log(err);  
    }  
});
```

Shorthand Methods

```
$.get("/api/users", function (data) {  
    console.log(data);  
});
```

Working with JSON & APIs

```
$.getJSON("/api/users", function (data) {  
  console.log(data);  
});
```

Use Case: Integrating backend APIs (.NET Web API)

Forms & Validation

```
$("#form").submit(function (e) {  
  e.preventDefault();  
  let name = $("#name").val();  
  if (!name) {  
    alert("Name required");  
  }  
});
```

Performance & Optimization

Optimization Tips

- Cache selectors
- Use event delegation
- Minimize DOM manipulation

```
let $btn = $("#btn");  
$btn.click(function () {});
```
